

KAN-IDS — consolidamento sperimentale

Protocollo leakage-free, validazione 5-fold × 3 seed, confronto con baseline identiche e cross-domain
TON_IoT ↔ BoT-IoT

Emanuele Pio De Bernardis — agosto 2026 — github.com/emanuelepiodebernardis/kan-ids-v2

1. Protocollo

Ogni trasformazione che apprende dai dati — ranking per mutual information, vocabolari delle categoriche, quantili — è ora fittata **esclusivamente sul training** di ciascun fold. Nella versione precedente il ranking era calcolato su un campione dell'intero dataset prima dello split, e i vocabolari categorici su train+test. La correzione vive in un solo modulo (*kanids/preprocessing.py*) usato da tutti gli script; due test automatici impediscono che il difetto rientri.

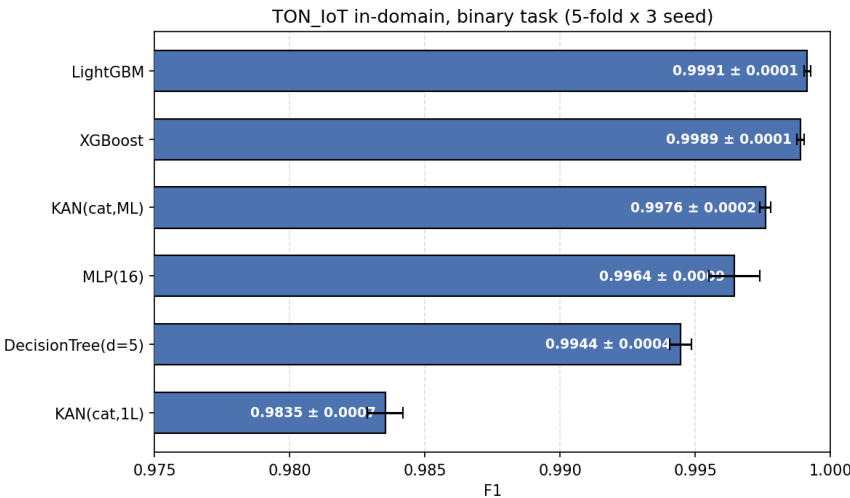
Effetto misurato del difetto: nullo. La selezione per-fold sceglie le stesse 10 feature numeriche in 15 fold su 15, e in-domain non esiste alcuna categoria assente dal training. I numeri precedentemente riportati restano quindi validi; il protocollo andava comunque corretto, perché la sua validità non può dipendere da quanto grande sia risultato l'effetto.

Validazione: **StratifiedKFold(5) ripetuta su 3 seed = 15 fit per modello**, media ± deviazione standard. La stratificazione usa sempre l'etichetta a 10 classi, così i modelli binari e multiclass vedono fold identici e la classe rara (MITM, 0,49% dei flussi) è presente ovunque.

2. Tabella comparativa — TON_IoT in-domain

Modello	F1 binario	PR-AUC	FPR	Macro-F1 10 classi	F1 MITM
LightGBM	0.9991 ± 0.0001	1.0000	0.0023	0.9680 ± 0.0021	0.767
XGBoost	0.9989 ± 0.0001	1.0000	0.0041	0.9666 ± 0.0021	0.761
KAN(cat,ML)	0.9976 ± 0.0002	0.9999	0.0037	0.9374 ± 0.0036	0.541
MLP(16)	0.9964 ± 0.0009	0.9998	0.0088	0.9182 ± 0.0107	0.386
DecisionTree(d=5)	0.9944 ± 0.0004	0.9981	0.0075	0.7633 ± 0.0033	0.151
KAN(cat,1L)	0.9835 ± 0.0007	0.9985	0.0208	0.8767 ± 0.0014	0.270

15 fit per modello e per task. Spazio a 14 feature (10 numeriche + 4 categoriche); tutti i modelli ricevono esattamente lo stesso input.



3. Cross-domain TON_IoT ↔ BoT-IoT

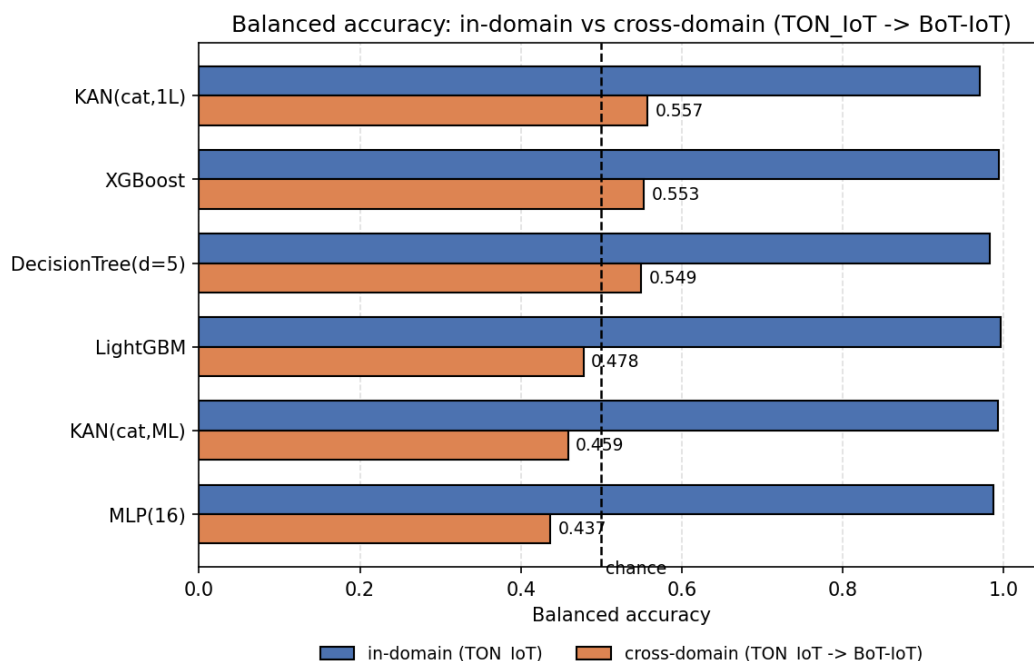
Task binario su uno **spazio armonizzato a 13 feature candidate**, calcolate con la stessa formula sui due dataset (durata, byte e pacchetti IP per direzione, totali, asimmetrie, payload medi, rate) più protocollo e stato mappati su un alfabeto semantico comune. Escluse porte e indirizzi (identificatori di testbed), gli aggregati a finestra di BoT-IoT e i metadati DNS/SSL/HTTP di TON_IoT. Nel cross-domain il target entra **solo** nella valutazione.

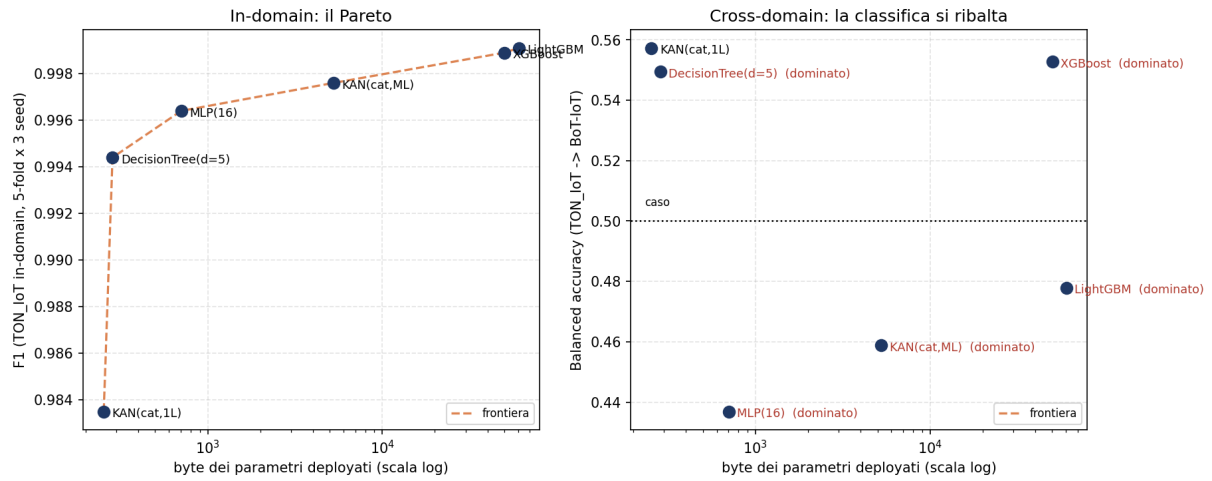
Nota sullo spazio di feature. In-domain lo spazio e' **10 numeriche + 4 categoriche** (proto, service, conn_state, dns_rejected). Nel cross-domain diventa **10 + 2**: delle quattro categoriche solo proto e stato hanno un corrispettivo nei log Argus di BoT-IoT, e includere le altre violerebbe il vincolo «solo feature realmente confrontabili». I due blocchi di risultati non sono quindi sullo stesso spazio, ed e' una conseguenza del vincolo, non una svista: la colonna in-domain della tabella cross-domain va letta come riferimento interno a quello spazio, non come confronto con la tabella precedente.

Nota sulle metriche. BoT-IoT è attacco al 99,987%: in quel regime la PR-AUC sulla classe positiva vale ~1 per costruzione e non discrimina (nei run TON→BoT è 0,9999 mentre i modelli sono al caso). Si riportano i due recall e la loro media, la balanced accuracy, dove **0,50 = caso**.

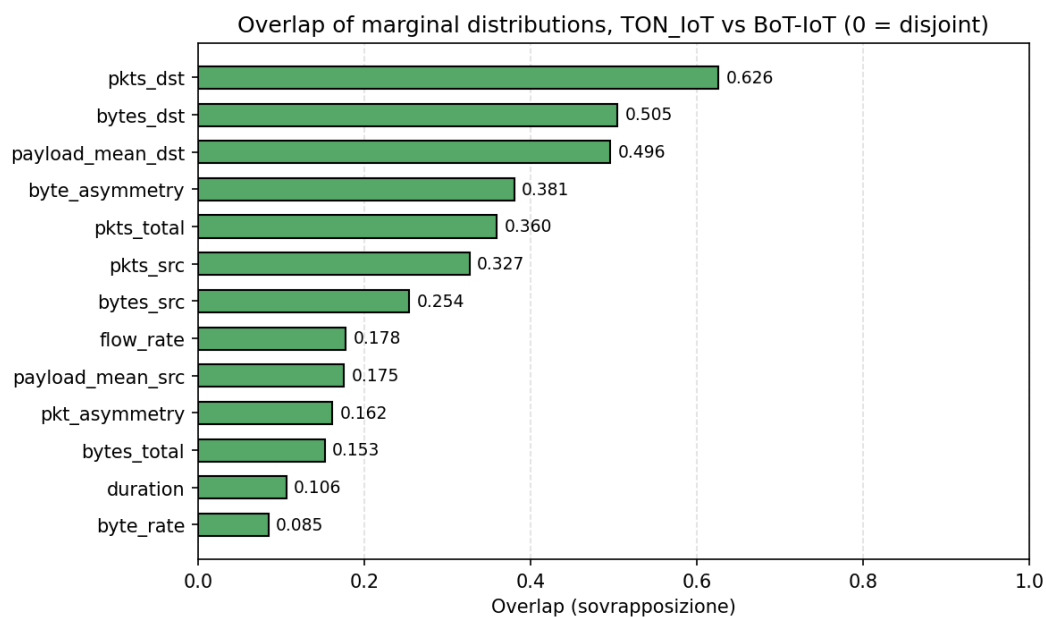
Modello	TON in-dom.	TON→BoT	δ	BoT in-dom.	BoT→TON	δ
MLP(16)	0.9879	0.4369	0.5510	0.9426	0.7343	0.2083
LightGBM	0.9963	0.4779	0.5184	0.9971	0.6964	0.3007
KAN(cat,ML)	0.9932	0.4588	0.5344	0.9971	0.6855	0.3116
XGBoost	0.9947	0.5528	0.4420	0.9779	0.6487	0.3292
KAN(cat,1L)	0.9701	0.5573	0.4128	0.9934	0.6112	0.3822
DecisionTree(d=5)	0.9828	0.5494	0.4334	0.9952	0.4597	0.5355

Balanced accuracy. In-domain: 50 fit (5 fold x 10 seed). Direzioni cross: 10 seed, training sull'intero source, valutazione sull'intero target.



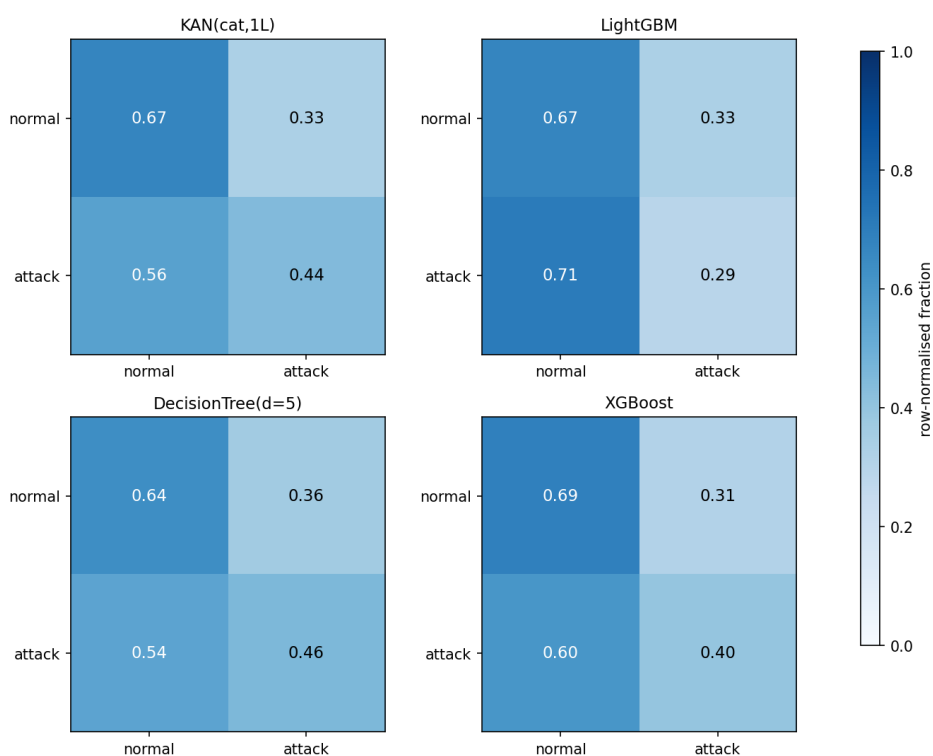


Frontiera di Pareto dimensione/accuratezza, byte contati sugli header C effettivamente compilati. A sinistra in-domain: il modello KAN da 254 byte è il più piccolo sulla frontiera, l'albero profondo 5 ne occupa 285 ed è più accurato: un compromesso, non una dominanza. A destra cross-domain: la classifica si ribalta e il single-layer è il modello che trasferisce meglio.



Sovrapposizione delle distribuzioni marginali fra i due domini. Valori sotto 0,2 significano supporti quasi disgiunti.

Confusion matrices, TON_IoT -> BoT-IoT (row-normalised)



Matrici di confusione TON_IoT → BoT-IoT, normalizzate per riga. LightGBM classifica come normale il 71% degli attacchi di BoT-IoT.

4. Analisi critica

4.1 Il confronto con le baseline era viziato

La versione precedente dichiarava che il modello da 254 byte supera gli ensemble ad alberi «sullo stesso spazio di feature deployabile». Non è così: quel confronto metteva la KAN sullo spazio grezzo a 14 feature e le baseline sullo spazio derivato a 10 feature del paper. Con input identici, **LightGBM raggiunge 0,9991 contro 0,9835 della KAN single-layer**, vincendo 15 fold su 15 (t-test appaiato $p = 1,0 \cdot 10^{-2}$). Il claim è stato corretto nel README.

4.2 La profondità colma il divario, e spiega perché esisteva

La KAN single-layer è un modello additivo generalizzato: somma di funzioni univariate, incapace per costruzione di rappresentare interazioni fra feature. La multi-layer, che può, guadagna **+0,0141 di F1 vincendo 15 fold su 15** e arriva a $0,9976 \pm 0,0002$. È la misura diretta che il divario riguardava le interazioni, non la capacità né l'ottimizzazione — e trasforma un punto debole in un risultato architetturale.

4.3 A parità di conteggio il Pareto è un compromesso, non una dominanza

Perché il confronto sia una misura e non un artefatto, i byte vanno contati con una regola sola, e deve essere quella che il codice implementa. Fino a questa revisione non lo era: footprint.py usava un impacchettamento ideale (4 byte per nodo interno, 1 per foglia) che l'albero in C non usa. L'header `mcu_pio/include/dt5_model.h` memorizza quattro array paralleli su tutti e 57 i nodi, foglie comprese, e occupa **285 byte**, non 141. Contati sugli header compilati (`scripts/c_footprint.py`, verificabili con `nm` sull'oggetto del compilatore), i due modelli più piccoli si invertono: la KAN single-layer sta in **254 byte**, l'albero in **285**.

Resta però vero che l'albero è **più accurato** (F1 0,9944 contro 0,9835) ed è invariante a trasformazioni monotone, quindi non richiede alcun preprocessing, mentre la catena integer end-to-end della KAN costa 1.334 byte comprese le tabelle. Il single-layer non è dominato, ma 31 byte di differenza sono un arrotondamento su entrambe le schede: l'argomento a favore del single-layer resta il comportamento cross-domain, non la dimensione. «Accuratezza per byte» non è un argomento difendibile su TON_IoT, e il repository non lo sostiene.

Tre cose invece reggono, e sono quelle su cui costruire. **(a)** La KAN multi-layer sta sulla frontiera: 5,12 KB e F1 0,9976 contro l'MLP TensorFlow Lite Micro del paper originale, 13 KB e 0,9959 con 95 feature — più piccola, più accurata, 14 feature invece di 95. **(b)** Nel cross-domain la classifica si ribalta: il single-layer ha il valore medio più alto su TON→BoT (0,5573), pur senza essere separabile da XGBoost né dall'albero — vedi `crossdomain_significativita.csv`; l'albero è a 0,5494 ed è il peggiore di tutti nella direzione BoT→TON (0,4597). **(c)** La KAN offre ciò che un albero non ha: calibrazione conformal sul modello intero deployato, forma simbolica chiusa e tabelle riscrivibili, che sono il presupposto della ricalibrazione on-device.

Con l'export in C dell'albero (`scripts/export_tree_c.py`) il confronto si può finalmente fare sul dispositivo, e un primo risultato c'è già: **quantizzare le soglie a Q7** per il target costa all'albero **0,0028 di F1** (0,9944 → 0,9916, agreement 99,55% col modello float). Il divario con la KAN compilata scende da 0,0109 a **0,0081**. La KAN paga la quantizzazione meno dell'albero — un argomento che prima non avevamo, perché confrontavamo un albero in virgola mobile con una KAN già quantizzata.

4.4 Il cross-domain non degrada: collassa

TON→BoT lascia ogni modello fra 0,44 e 0,56 di balanced accuracy, cioè al caso o sotto. Il δ quantificato nel paper era $\leq 5,95$ punti; qui siamo a **41–55 punti**. Due osservazioni non ovvie: il compromesso fra capacità e trasferibilità si misura **dentro la stessa famiglia** — aggiungere profondità alla KAN vale +0,0141 di F1 sul binario e +0,0608 di macro-F1 sulle 10 classi, e le costa quasi tutto il transfer (0,4588 di balanced accuracy, **sotto il caso**), mentre la single-layer, ultima in-domain, ha il valore più alto cross-domain. Il modello che trasferisce peggio non è però la KAN profonda ma MLP(16) (0,4369): a 3

seed sembrava il contrario, ed è una delle affermazioni che i 10 seed hanno ritirato. E la direzione BoT→TON non è degradata ma **indeterminata** — con 477 flussi normali in training la varianza fra seed supera la differenza fra modelli, quindi qualunque classifica in quella direzione sarebbe rumore.

La causa è visibile prima di addestrare qualsiasi cosa: le marginali non si sovrappongono (byte_rate 0,085, duration 0,106). TON_IoT ha flussi brevi e bidirezionali, il 5% di BoT-IoT è dominato da flood UDP lunghi e unidirezionali. Il 21,3% dei flussi di TON_IoT porta uno stato di connessione mai visto addestrando su BoT-IoT. Gli edge categorici armonizzati aiutano il transfer nella maggior parte dei casi ma non sempre: su 11 celle misurate in entrambe le varianti rimuoverli sposta la balanced accuracy fra -0,2519 e +0,1859, e in 2 celle la rimozione aiuta. Una versione precedente di questo paragrafo dava «0,08–0,16», un intervallo che escludeva proprio le celle che lo contraddicono.

4.5 La catena integer end-to-end ora è in C, e ha rivelato tre difetti

Il percorso dai contatori grezzi alla decisione gira interamente in aritmetica intera nel firmware: **200 golden vector su 200 con logit bit-identico** al riferimento Python, e l'ispezione dell'assembly del percorso di inferenza mostra **zero istruzioni in virgola mobile**. Il percorso end-to-end precedente (*mcu_e2e*) interpolava 10.000 knot del QuantileTransformer in doppia precisione: end-to-end nella struttura, ma non integer-only nel runtime. Il porting ha fatto emergere tre difetti che si sarebbero manifestati solo sul dispositivo: il moltiplicatore di scala della feature con scala massima quantizza a esattamente 32768, che in *int16* va in overflow silenzioso; la divisione intera di Python arrotonda verso $-\infty$ mentre quella del C tronca verso zero, differenza osservabile sulle due feature di asimmetria, che hanno numeratore di segno qualsiasi; e la stessa saturazione a 2^{15} ricompare nella LUT della tangente iperbolica del modello a 10 classi.

Anche la **catena a 10 classi** è ora in C: contatori grezzi → ricerca binaria sulle soglie per-feature → *z* in *Q12* → primo strato di spline *int8* con tabelle categoriche → LUT *tanh* → secondo strato → *argmax*, tutto intero. **200 golden vector su 200 con tutti e dieci gli accumulatori bit-identici** al riferimento, *argmax* identico, macro-F1 0,9352 contro 0,9378 della pipeline float (agreement 99,42%), 21,7 KB di tabelle. Le soglie restano a 64 bit perché su TON_IoT *src_bytes* e *dst_bytes* arrivano a $3,9 \cdot 10^9$: sono i contatori di byte a imporre i 64 bit, non la durata.

La catena è anche **deployata**, non solo verificata: *mcu_pio/src/main_e2e.cpp* parte dai contatori grezzi ed esegue a bordo l'intera pipeline, feature engineering compreso, sotto il protocollo di benchmark del paper. Tutte le altre varianti di firmware ricevono vettori già normalizzati fuori dal dispositivo: senza questa, la catena sarebbe dimostrata corretta ma non sarebbe mai stata la pipeline in esecuzione sull'MCU. Firmware e harness di verifica includono lo stesso kernel (*include/kan_e2e_infer.h*), quindi ciò che è verificato bit per bit è ciò che gira sulla board, non una copia che può divergere.

4.6 Sul multiclass la profondità pesa quattro volte di più

Con lo stesso protocollo sulle 10 classi la KAN multi-layer fa **$0,9374 \pm 0,0036$** contro 0,8767 della single-layer: **+0,0608 di macro-F1, 15 fold su 15**, contro il +0,0141 del binario. Separare le famiglie di attacco richiede interazioni fra feature molto più di quanto ne richieda separare attacco da normale — stesso argomento strutturale, effetto quattro volte più grande. E l'albero profondo 5, che sul binario era il modello più piccolo e più accurato, qui crolla all'ultimo posto, 0,1741 sotto la multi-layer: il suo dominio in-domain era specifico del problema binario e non sopravvive al task più difficile.

4.7 Il soffitto su MITM è informativo, e riguarda tutti

Sul multiclass la classe MITM (1.043 flussi, 0,49%) è il collo di bottiglia per **ogni** modello: LightGBM 0,767, MLP 0,386, KAN single-layer 0,270, albero profondo 5 0,151. Per ogni modello tranne l'albero profondo 5 tutte le altre classi stanno sopra 0,88. Nessuno dei sei modelli supera 0,77 su MITM, e né focal loss né SMOTENC né il class weighting l'hanno spostata: è coerente con un limite dell'informazione disponibile in questo spazio di feature, ma non lo dimostra — sei architetture non esauriscono lo spazio delle architetture, e la dispersione fra loro su questa classe (da 0,151 a 0,767, un fattore cinque) è la più larga di

ogni altra classe, quindi qui l'architettura pesa ancora molto.

4.8 La stima riportata non è ottimista: è conservativa

La cross-validation è corretta per una pipeline fissata, ma la nostra non lo era: $k=10$ è stato scelto guardando risultati calcolati sugli stessi dati. Invece di argomentare che l'effetto fosse piccolo, l'ho misurato con una **cross-validation annidata** — la scelta di k avviene dentro ogni fold esterno, su dati che non partecipano alla valutazione.

Risultato: l'ottimismo è **negativo**. La stima annidata vale $0,9845 \pm 0,0006$ contro lo 0,9835 riportato per la KAN single-layer ($-0,0009$), e 0,9992 contro 0,9991 per LightGBM ($-0,0001$). I numeri pubblicati sono semmai leggermente conservativi, perché il protocollo piatto è vincolato a un $k=10$ ereditato mentre quello annidato è libero di scegliere meglio.

La misura però smentisce un'altra affermazione. La selezione interna **non sceglie mai $k=10$** : prende tutte e 16 le feature candidate in 15 fold su 15 per la KAN. La curva media è monotona, non ha un picco: 0,9795 a $k=5$, 0,9836 a $k=10$, 0,9845 a $k=16$. Il claim «l'accuratezza ha il massimo esattamente a 10 feature» non sopravvive al protocollo corretto. **$k=10$ è una scelta di deployment** — dieci statistiche di flusso da calcolare a bordo invece di sedici — e ora ha un prezzo misurato: 0,0009 di F1. È un argomento migliore di un picco che non c'è.

4.9 Cosa si può effettivamente flashare

Ogni modello esportato ha un firmware e un environment PlatformIO — **sette su sette** — quindi ognuno è misurabile sulle due board con lo stesso protocollo: KAN LUT intera, KAN single-layer a coefficienti, KAN multi-layer, KAN multiclass, catena end-to-end binaria, catena end-to-end a 10 classi e Decision Tree profondo 5. Prima alcuni esistevano come header ma senza un *main* che li usasse: esportati sulla carta, non testabili fisicamente. Un test lo impedisce ora, e ne compila sette su sette a ogni esecuzione della suite, senza bisogno di toolchain per MCU.

Nello stesso controllo è emerso che l'export in C del modello multiclass era rimasto al **protocollo v1**: tabelle categoriche a 28 righe invece di 32, cioè senza lo slot UNK. Il firmware girava su un modello incompatibile con il preprocessing attuale, e senza errori visibili perché modello e vettori di test erano coerenti fra loro. Rigenerato; un test vieta ora qualunque header con tabelle a 28 righe.

5. Stato e passi successivi

Punto	Stato
1. Protocollo leakage-free	completato, con misura dell'effetto
2. Multi-layer con lo stesso protocollo	completato: $0,9976 \pm 0,0002$
3. BoT-IoT, 4 direzioni	completato, con analisi del degrado
4. Baseline identiche	completato su binario e multiclass
6. Riproducibilità da clone pulito	completato, verificato
5. Integer-only end-to-end (binario)	completato: 200/200 bit-esatti, 1.334 B
5. Integer-only end-to-end (10 classi)	completato: 200/200 bit-esatti, 21,7 KB
5. Firmware che parte dai contatori grezzi	completato: main_e2e.cpp, 200 golden vector verificati, 500 inferenze temporizzate
Ogni modello esportato in C e flashabile	completato: 8 firmware, 21 environment PlatformIO
Benchmark di energia a batch, senza I/O nella finestra	completato: main_energy.cpp, 9 environment; misure sulle schede a cura del relatore
Coerenza degli artefatti deployati	completato: multiclass rigenerato al protocollo v2
Ingombro dei parametri (asse dimensione)	misurato: results/footprint.csv
models/ + manifest (protocollo, seed, metriche)	completato
Conformal e forma simbolica sotto v2	rigenerati: coperture 99,05/94,90/90,23
Benchmark fisici (latenza, energia, code size)	da fare — richiedono le board
Focal loss e SMOTENC (risultati negativi)	ancora v1; corroborati indipendentemente in v2
Multi-layer multiclass in CV 5x3	completato: $0,9374 \pm 0,0036$ su 15 fit
Indipendenza della stima (CV annidata)	misurata: ottimismo -0,0009, conservativo
CIC-IoT-2023 come terzo dataset	non iniziato (obiettivo secondario)

La priorità che suggerirei è il benchmark fisico: senza gli ingombri misurati, il confronto con l'albero profondo 5 resta aperto ed è la prima obiezione che farebbe un revisore. Il cross-domain, dal canto suo, fornisce la motivazione quantificata al passo successivo: a 40 punti di gap l'adattamento on-device non è un miglioramento marginale, ma la condizione perché un IDS embedded funzioni fuori dal laboratorio in cui è stato addestrato.